

=====

===== ADVANCED NETWORK THREAT DETECTION & TELEMETRY ANALYSIS LAB OVERVIEW

=====

===== Target Career Alignment: SOC Analyst / Security Engineer Author / Lab Architect : Jack Jusko Date of Architecture : July 2026

----- 1. EXECUTIVE EXECUTIVE SUMMARY & CORE OBJECTIVES

----- The primary objective of this engineering lab is to design, deploy, and triage an enterprise-grade Network Threat Detection system within a completely isolated virtual sandbox environment. By combining an open-source Intrusion Detection System (Suricata) with a centralized Security Information and Event Management (SIEM) platform (Wazuh), this architecture establishes full network visibility and security analytics across the wire. Key Objectives: * Validate Network Visibility: Implement software-defined promiscuous mode over a virtual broadcast domain to bypass hardware-level packet filtering. * Out-of-Band Management Design: Deploy a dual-homed security sensor to segregate dirty, high-risk attack traffic from critical SIEM reporting pipelines. * Signal-to-Noise Engineering: Analyze high-fidelity signature alerts and witness the physical performance boundaries of endpoint log transmission agents. * Offensive Triage: Simulate real-world reconnaissance behaviors (Layer 4 TCP stealth scanning) to dissect how network footprints translate to structured JSON telemetry alerts.

----- 2. LAB NETWORK TOPOLOGY & IP SCHEMA

----- The network is hard-configured to utilize the private, non-routable RFC 1918 Class C space to completely isolate malicious scans from the physical home LAN. Subnet Scope: 192.168.10.0/24 (Subnet Mask: 255.255.255.0) Nodes Matrix:

Host Node	NIC 1 (Lab Switch)	NIC 2 (Management)	Operational Role
192.168.10.5	Disconnected	Threat Actor / Simulation	Suricata IDS Host
192.168.10.10	DHCP (Active WAN)	Network Security Sensor	Ubuntu Victim VM
192.168.10.20	Disconnected	Target Web Infrastructure	

+-----+ Architectural Design Constraints: * Promiscuous Mode: Enabled globally on the virtual switch segment. Forces the Suricata host network card to pass all packets traveling across the switch up to the detection engine, rather than dropping frames not explicitly addressed to its own MAC address. * Dual-Homing: The Suricata IDS VM is deliberately provisioned with two distinct Network Interface Cards (NICs). NIC 1 acts as a "blind sniffer" on the internal switch with no default gateway. NIC 2 serves as a dedicated out-of-band management pipeline to stream security events cleanly to the master Wazuh Manager without routing logs through an active attack path.

----- 3. PHASE-BY-PHASE IMPLEMENTATION BLUEPRINT

PHASE I: PRE-ISOLATION DEPENDENCY STAGING * Target Server: Provisioned a base Apache2 web server framework and traditional network utility toolsets (net-tools) on the Victim

node while internet access was active. * Attacker Station: Updated the local Kali application suite to ensure Layer 7 enumeration scripts (Gobuster) and standard directory brute-force wordlists (dirb common list) were locally cached on disk. * Security Sensor: Installed Suricata alongside the command-line JSON engine (jq). * Threat Intelligence Hydration: Executed live synchronization rules to pull and compile the latest Emerging Threats Open signature mapping bundle to disk. * Log Pipe Integration: Installed and registered the centralized SIEM monitoring agent daemon on the sensor operating system. PHASE II: NETWORK ISOLATION & LINK CONTROL * Severed WAN Connections: Cable links for the internet-facing adapters on both the Kali Attacker and Victim nodes were dynamically disconnected within the hypervisor runtime layer to prevent malicious script leakage. * Static Interface Pinning: Swapped local OS configuration states from dynamic DHCP calls to persistent static variables matching the internal lab subnet schema. * Promiscuous Interface Activation: Issued raw kernel commands on the Suricata node to force the sniffing interface card into absolute listening mode. PHASE III: INTRUSION ENGINE TUNING & TELEMETRY PIPELINES * Subnet Identification: Reconfigured the master variable definitions file (suricata.yaml) to restrict internal scanning tracking exclusively to the custom 192.168.10.0/24 subnet. * Engine Thread Binding: Mapped Suricata's multi-threaded packet processing loops natively onto the validated interface capturing internal traffic frames. * SIEM Pipeline Connection: Configured the local log agent (ossec.conf) to actively ingest and tail Suricata's unified JSON output stream (eve.json). * Client Buffer Tuning: Handled queue congestion anomalies during heavy traffic volume by adjusting client-side buffer constraints to bypass validation loops and prevent the agent from dropping critical security events during automated floods.

----- 4. SIMULATION EXECUTION
VECTORS (THE ATTACK) ----- With the network containment active and the logging pipeline fully established, two core adversarial techniques are fired from the Kali platform: Vector A: Layer 4 Network Reconnaissance (Nmap Stealth Scan) * Command: `nmap -sS -A -v 192.168.10.20` * Concept: Sends raw TCP packets with the SYN flag activated without completing the three-way handshake. Compares the subtle TCP window properties and packet responses from the victim machine against an internal catalog to reverse-engineer the host operating system type and map active listener ports. Vector B: Layer 7 Directory Enumeration (Web Fuzzing Application Probe) * Command: `gobuster dir -u http://192.168.10.20 -w /usr/share/wordlists/dirb/common.txt` * Concept: Automated application-layer fuzzing that hammers the target web server with consecutive rapid-fire HTTP GET transactions mapping to common directory paths, identifying hidden administrative interfaces based on response code metrics.

----- 5. SIEM TRIAGE & TELEMETRY
EVIDENCE ANALYSIS ----- When reviewing the centralized dashboard, an enterprise security analyst isolates and correlates indicators of compromise using the following telemetry criteria: * High-Fidelity Network Alerts: Filtering the SIEM data array using dedicated queries (rule.groups: "suricata") surfaces distinct signature rule triggers generated by the raw Nmap scan, pointing to anomalous flag combinations and explicit OS identification attempts. * Agent Queue Metrics: Monitoring localized log health metrics uncovers the consequences of aggressive, unthrottled directory busters—manifesting as "Agent event queue is full" alerts on the dashboard when local internal

software buffers are saturated by the volume of raw events. * Forensic Metadata Artifacts: Evaluating the deeper, raw JSON objects nested inside the Suricata alert scheme allows an investigator to inspect transport protocol states, verify specific attacker-controlled source IPs, and evaluate signature IDs (SIDs) derived from the open-source intelligence feed.

=====

===== END OF LAB OVERVIEW DOCUMENT =====

=====

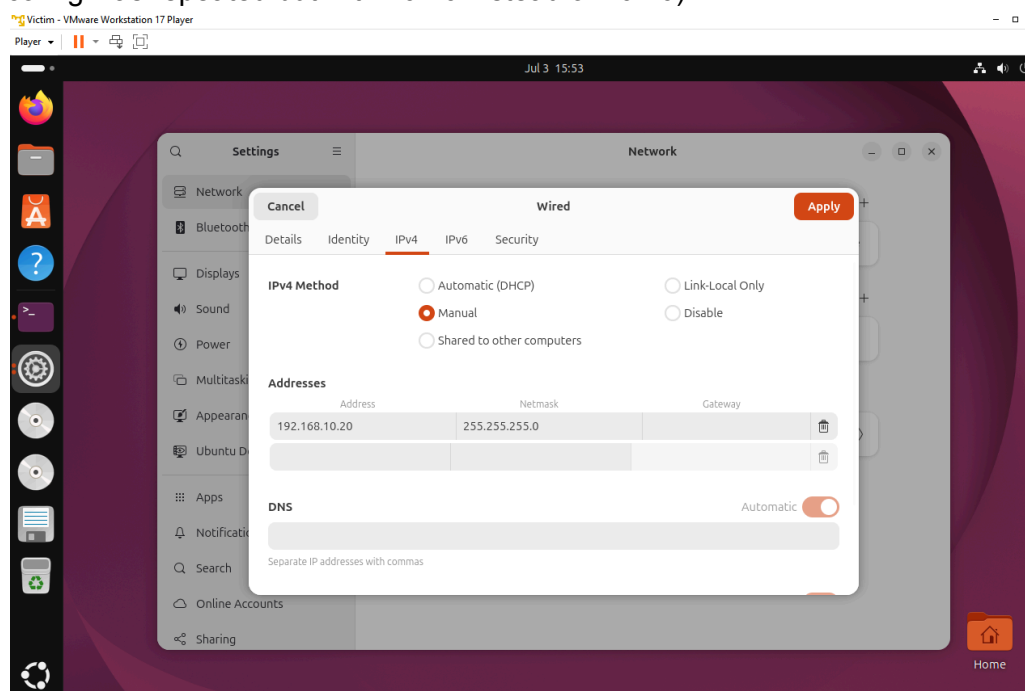
Goal: test identify and defend against directory enumeration

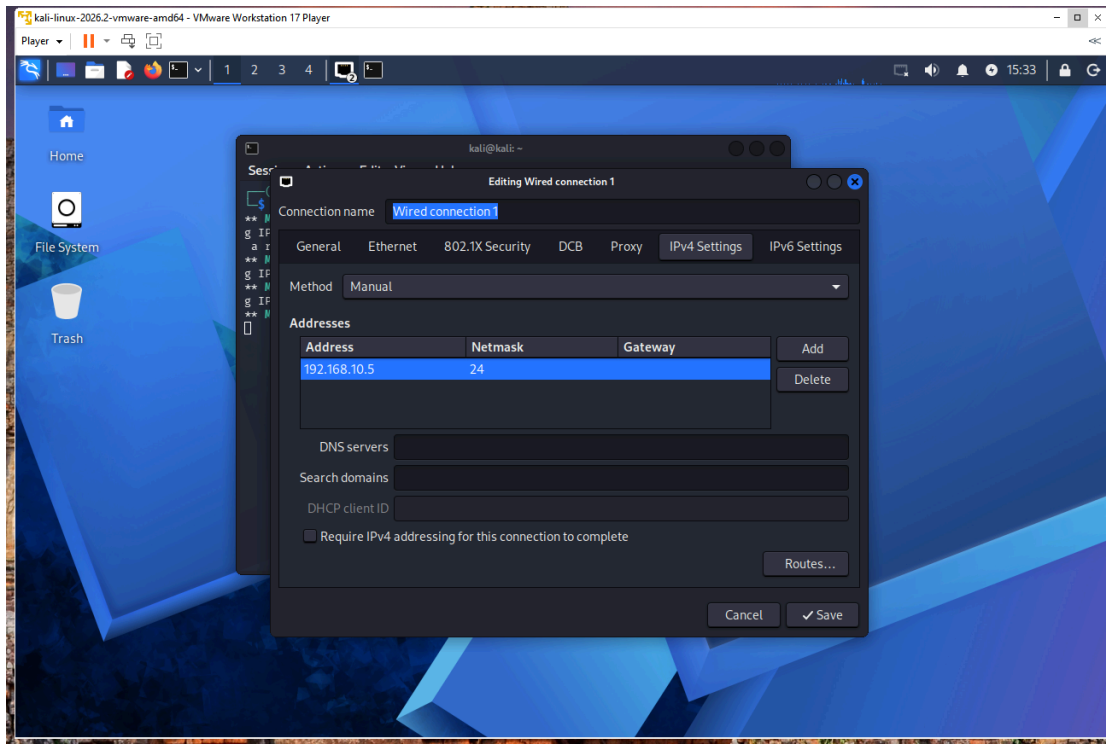
This lab integrated suricata logging into the wazuh installation detailed in my previous lab: the brute forcing procedure lab.

Machines: 2 unique ubuntu 24.04 boxes (one for suricata, one victim) and one kali linux box, all on vmware

Step 1: configure virtual network switch for all devices

(I connected all three machines to the same lan segment through VMWare workstation player 17 and manually configured all IPs) (the two screenshots below show me assigning the networking for the victim VM and the attacker VM, respectively, for the suricata VM the ubuntu config was repeated but with 10.10 instead of 10.20)



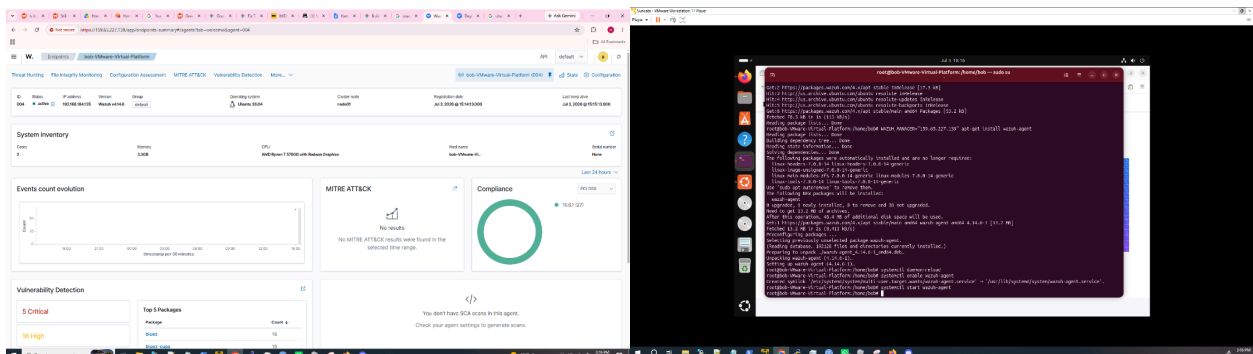


After this I installed and started apache on the victim machine - I was forced to temporarily revert the network changes to do the download and install for apache. After considering, I opted to run two virtual adapters (one for the LAN segment for this simulation, and one NAT adapter for package installations) per VM.

“I designed my network lab architecture to mirror enterprise best practices by dual-homing the Suricata sensor” – basically, each VM gets a NAT interface to the public web and a LAN segment for connections - I can download the packages I need for the simulation to each VM and then disable the network adapters, and leave only the suricata VM connected to a wazuh agent

I installed apache on the victim’s server

After all this, I installed suricata on my network monitor VM and updated the threat signatures. The network monitor VM also received a wazuh agent pointing to the cloud wazuh manager

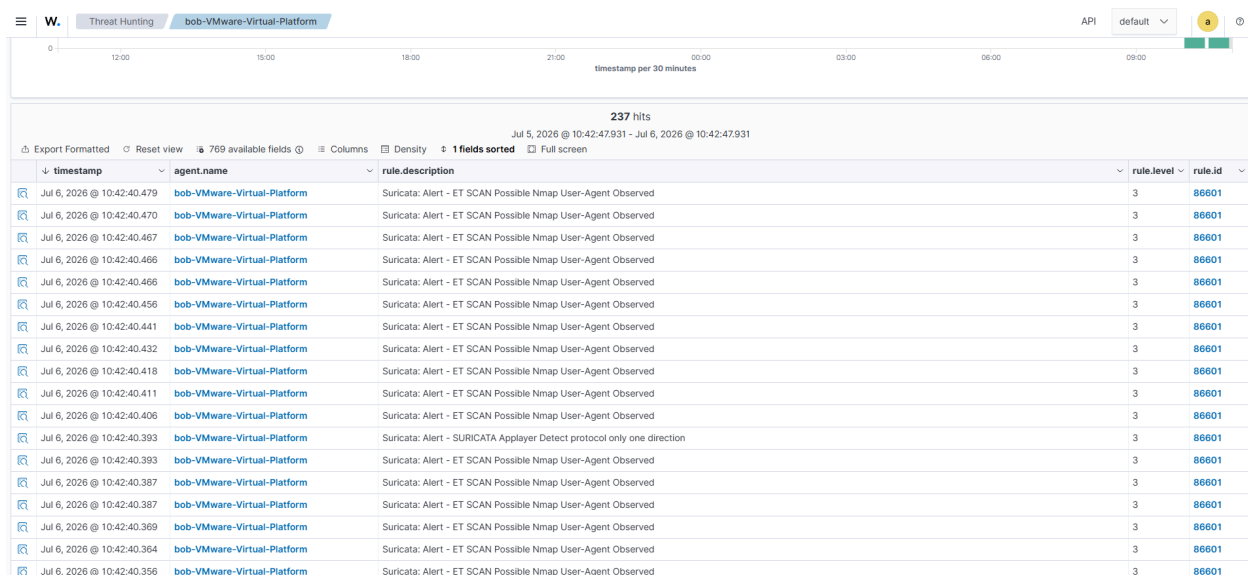


The above image is a screenshot of my home machine showing a wazuh install on the suricata VM

I set the suricata monitor to target the network adapter for my lan segment, and set it to the proper subnet that these virtual devices are contained on (192.168.10.0/24 was the subnet, ens33 was the adapter)

I did the step “Tell the local Wazuh agent to monitor Suricata’s output stream by opening its configuration: “ by adding the output stream json file to my local wazuh agent’s ossec.conf

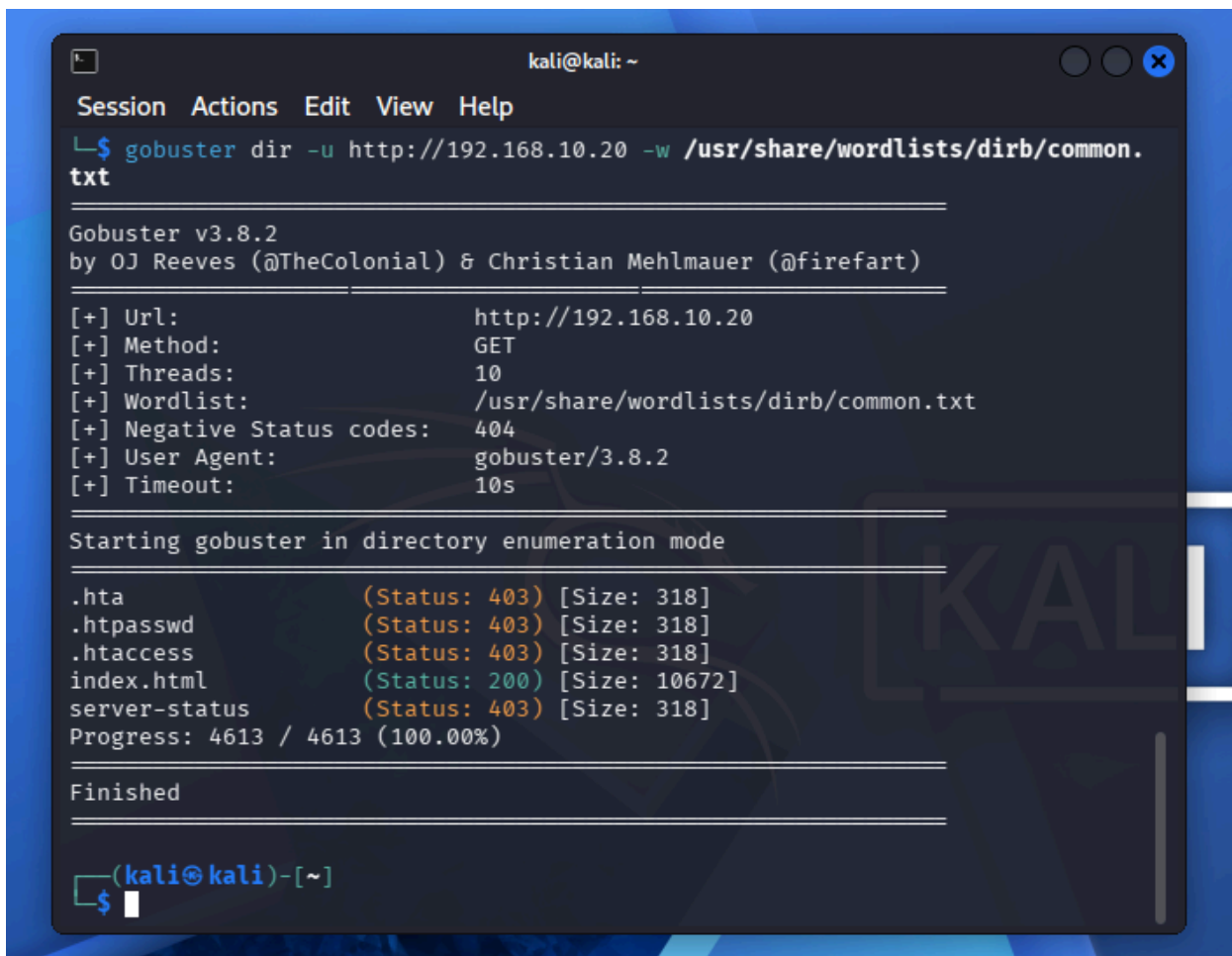
I ran a network scan using nmap via kali linux and got the alerts in wazuh via the suricata monitoring VM



timestamp	agent.name	rule.description	rule.level	rule.id
Jul 6, 2026 @ 10:42:40.479	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.470	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.467	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.466	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.466	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.456	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.441	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.432	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.418	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.411	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.406	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.393	bob-VMware-Virtual-Platform	Suricata: Alert - SURICATA Applayer Detect protocol only one direction	3	86601
Jul 6, 2026 @ 10:42:40.393	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.387	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.387	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.369	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.364	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Jul 6, 2026 @ 10:42:40.356	bob-VMware-Virtual-Platform	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601

The above is a screenshot showing the suricata logs detecting the nmap scan appearing in my wazuh instance

After that, I ran a gobuster directory enumeration attack on the apache client running on the victim machina, via gobuster on the kali machine



```
kali@kali: ~  
Session Actions Edit View Help  
└─$ gobuster dir -u http://192.168.10.20 -w /usr/share/wordlists/dirb/common.  
txt  
=====
```

Gobuster v3.8.2		
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)		
=====		
[+] Url:	http://192.168.10.20	
[+] Method:	GET	
[+] Threads:	10	
[+] Wordlist:	/usr/share/wordlists/dirb/common.txt	
[+] Negative Status codes:	404	
[+] User Agent:	gobuster/3.8.2	
[+] Timeout:	10s	
=====		

```
Starting gobuster in directory enumeration mode  
=====
```

.hta	(Status: 403)	[Size: 318]
.htpasswd	(Status: 403)	[Size: 318]
.htaccess	(Status: 403)	[Size: 318]
index.html	(Status: 200)	[Size: 10672]
server-status	(Status: 403)	[Size: 318]

```
Progress: 4613 / 4613 (100.00%)  
=====
```

```
Finished  
=====
```

```
(kali@kali)-[~]  
└─$
```

The above image is a screenshot of my kali instance showing the result of the gobuster attack

Ran into an issue here where the wazuh agent event queue was overloading under the volume of the brute force attacks - I changed the wazuh agent queue size to 80,000 and the events per second to 1000 (from 5000 and 500 respectively) and ran the gobuster scan again, after restarting the wazuh agent. I continued to have issues with the wazuh agent, so I opted to disable the client buffer entirely for this simulation.

TableJSON

† _index	wazuh-alerts-4.x-2026.07.06
† agent.id	004
† agent.ip	192.168.184.135
† agent.name	bob-VMware-Virtual-Platform
⌚ data.alert.action	allowed
⌚ data.alert.category	Web Application Attack
⌚ data.alert.gid	1
⌚ data.alert.metadata.affected_product	Any
⌚ data.alert.metadata.attack_target	Client_and_Server
⌚ data.alert.metadata.confidence	Medium
⌚ data.alert.metadata.created_at	2017_06_08
⌚ data.alert.metadata.deployement	Perimeter
⌚ data.alert.metadata.performance_impact	Low
⌚ data.alert.metadata.reviewed_at	2024_05_06
⌚ data.alert.metadata.signature_severity	Informational
⌚ data.alert.metadata.updated_at	2024_03_07
⌚ data.alert.rev	5
⌚ data.alert.severity	1

The above is a screenshot showing a wazuh event detailing a portion of an nmap scan pickup, with a signature of “

ET SCAN Possible Nmap User-Agent Observed

“